

Time Series Forecasting using Recurrent Neural Networks

Willie Loftie-Eaton

Computer Science Division, Stellenbosch University

Stellenbosch, South Africa

Email: 24717274@sun.ac.za / willieloea@gmail.com

Index Terms—Recurrent Neural Networks

I. INTRODUCTION

II. BACKGROUND

A. Neural networks

An artificial neural network learns a mapping from input features to outputs using interconnected processing units called neurons. In a feedforward network, information passes from an input layer through one or more hidden layers to an output layer. Each processing neuron forms a weighted sum of its inputs, adds a bias, and applies an activation function [1, Sec. 8.2]:

$$a = \phi \left(\sum_{i=1}^m w_i x_i + b \right), \quad (1)$$

where x_i are the inputs, w_i are connection weights, b is the bias, and ϕ is the activation function. Nonlinear hidden-layer activations allow the network to represent nonlinear relationships; stacking purely linear layers would still produce a linear mapping, apart from the bias terms.

During supervised training, a forward pass produces a prediction, which is compared with the known target using a loss function. Backpropagation applies the chain rule to calculate the loss gradients with respect to the weights and biases. An optimisation algorithm, such as gradient descent, then uses these gradients to adjust the parameters. Repeating this process over training examples allows the network to learn the input–output relationship [1, Sec. 8.3]. The output layer depends on the task: numerical forecasting can use a single output unit with a linear activation.

B. Recurrent neural networks

Recurrent neural networks (RNNs) process ordered sequences using feedback connections that carry information between time steps. The same weights are reused at each step, while the network’s state changes as new observations are processed. Consequently, a prediction can depend on earlier observations as well as the current input. This state acts as a learned summary of the sequence, rather than an explicit copy of every previous observation [1].

In an *Elman RNN* [2], context units retain the previous hidden-layer activations. These are supplied through weighted

connections to the hidden layer alongside the current input. A simple state update is

$$\mathbf{h}_t = \phi(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}_h), \quad (2)$$

where $\mathbf{x}_t$ is the current input and $\mathbf{h}_{t-1}$ is the previous hidden state. The matrices W_x and W_h and bias $\mathbf{b}_h$ are learned parameters. Because the previous state depends on earlier states, information can influence predictions beyond a single time step.

A *Jordan RNN* [3] instead feeds output activations back through context units to the hidden layer. Some Jordan formulations also include self-connections in the context units, allowing older context to persist.

A combined *multi-recurrent* architecture can incorporate both hidden-state and output feedback. These feedback signals come from earlier time steps; the unknown target being predicted is not available as an input. The precise context update must be specified when defining each architecture.

RNNs can be trained using backpropagation through time, which follows the dependencies across the sequence to calculate gradients. Gradients may shrink or grow excessively over long sequences, making long-term dependencies difficult to learn.

C. Cross-validation for time series data

Cross-validation estimates how well a model generalises to unseen observations and supports the selection of models and hyperparameters. Standard shuffled k -fold cross-validation is unsuitable for forecasting because it can place future observations in the training set while evaluating predictions for the past. Time-series validation instead preserves chronological order: each model is trained on earlier observations and evaluated on a subsequent period.

In walk-forward validation, the evaluation period moves forward between folds. An *expanding window* retains all preceding training observations, whereas a *sliding window* retains a fixed amount of recent history and discards older observations. Expanding windows make use of increasing amounts of data; sliding windows can be useful when older patterns become less representative. Errors across successive validation periods provide a broader assessment than a single split, and the forecast horizon should match the intended prediction task.

A *chronological holdout* reserves a final period for evaluation after model selection. Although this is not itself cross-validation, it complements the repeated validation used for tuning. To

prevent information leakage, learned preprocessing operations, such as scaling, must be fitted using only the training portion of each fold. The final holdout must not influence hyperparameter selection.

III. METHODOLOGY AND IMPLEMENTATION

IV. EMPIRICAL PROCEDURE

A. Datasets

The Mackey-Glass equation [4] is often used for comparing RNNs [5] [6] [7].

V. RESULTS AND DISCUSSION

VI. CONCLUSION

REFERENCES

- [1] J. D. Kelleher, B. Mac Namee, and A. D'Arcy, *Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies*, 2nd ed. MIT Press, 2020.
- [2] J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/036402139090002E>
- [3] M. I. Jordan, "Serial order: A parallel distributed processing approach," *Advances in psychology*, vol. 121, pp. 471–495, 1997. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15375627>
- [4] M. C. Mackey and L. Glass, "Oscillation and chaos in physiological control systems," *Science*, vol. 197, no. 4300, pp. 287–289, 1977. [Online]. Available: <https://www.science.org/doi/abs/10.1126/science.267326>
- [5] A. Logar, E. Corwin, and W. Oldham, "A comparison of recurrent neural network learning algorithms," in *IEEE International Conference on Neural Networks*, 1993, pp. 1129–1134 vol.2.
- [6] S. Shahi, F. H. Fenton, and E. M. Cherry, "Prediction of chaotic time series using recurrent neural networks and reservoir computing techniques: A comparative study," *Machine Learning with Applications*, vol. 8, p. 100300, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666827022000275>
- [7] R. Couillet, G. Wainrib, H. T. Ali, and H. Sevi, "A random matrix approach to echo-state neural networks," in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 517–525. [Online]. Available: <https://proceedings.mlr.press/v48/couillet16.html>